

Aula Prática 10 – Identificação de Genes de Virulência em Bactérias

Nicholas Barroso

2026-07-18

Índice

1	Introdução	2
2	Objetivos	2
3	Fluxograma	4
4	Parte 1 – Montagem do Genoma	5
4.1	Controle de Qualidade	5
4.2	Montagem com SPAdes	6
4.3	Verificação da Montagem	6
4.4	Visualização da Montagem	7
5	Parte 2 – Predição de Genes (30 minutos)	7
5.1	Predição com Prokka	7
5.2	Visualização da Predição	8
6	Parte 3 – Anotação Funcional (30 minutos)	9
6.1	Anotação com DIAMOND + Uniprot	9
6.2	Análise de Anotação Funcional	10
6.3	Identificação de Genes de Virulência	11
7	Parte 4 – RNA-seq (45 minutos)	12
7.1	Alinhamento e Quantificação	12
7.2	Análise de Expressão Diferencial	13
8	Parte 5 – Coexpressão Gênica (45 minutos)	15
8.1	Rede de Coexpressão com WGCNA	15

8.2	Associação com Fenótipos	16
9	Parte 6 – Rede PPI (30 minutos)	17
9.1	Construção da Rede PPI	17
10	Parte 7 – Integração dos Resultados (30 minutos)	18
10.1	Integração Multi-ômica	18
10.2	Geração de Relatório	19
11	Resumo Final	20
11.1	Pipeline Completo (scripts)	20
11.2	Resultados Principais	20
11.3	Figuras Geradas	20
11.4	Interpretação Biológica Final	21
11.5	Principais Correções Realizadas	21
11.6	Como Publicar no GitHub Pages	21

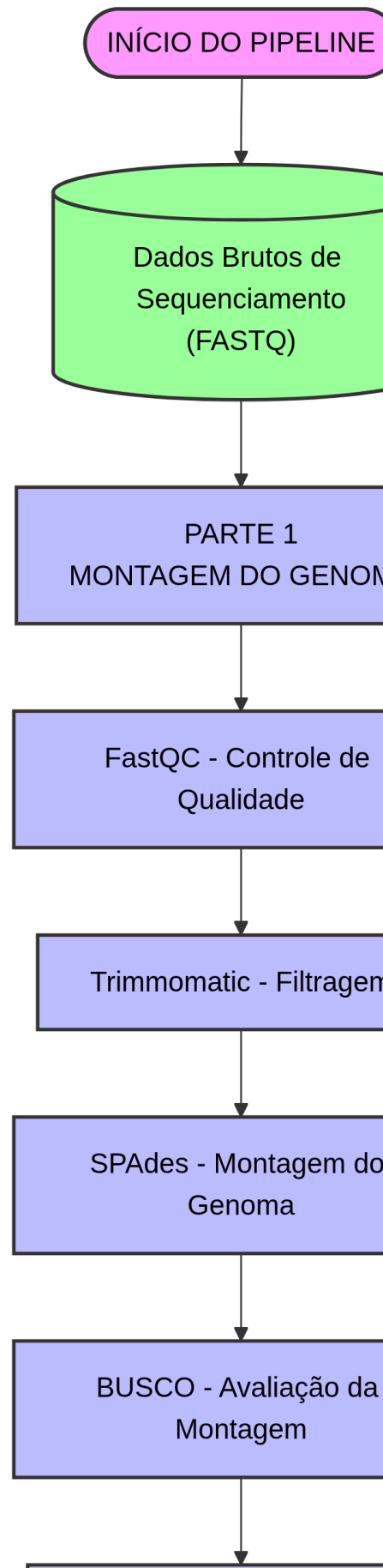
1 Introdução

A identificação de genes de virulência em bactérias é interessante para compreender os mecanismos de patogenicidade. Este roteiro apresenta um pipeline que tenta integrar abordagens ômicas para descobrir genes envolvidos na virulência bacteriana.

2 Objetivos

1. **Montagem e anotação** do genoma bacteriano para identificar todos os genes presentes
2. **Análise de expressão** para identificar genes diferencialmente expressos em condições virulentas
3. **Coexpressão gênica** para detectar módulos funcionais associados à virulência
4. **Redes de interação** para identificar genes centrais (hubs) e suas conexões
5. **Integração multi-ômica** para priorizar genes candidatos com múltiplas evidências

3 Fluxograma



4 Parte 1 – Montagem do Genoma

4.1 Controle de Qualidade

```
#!/bin/bash

# scripts/01_assembly.sh

# Diretórios
DATA_DIR="../../data/raw_reads"
RESULTS_DIR="../../results/assembly"
mkdir -p $RESULTS_DIR

# 1. FastQC - Controle de qualidade
echo "<mark>= FASTQC </mark>="
fastqc $DATA_DIR/*.fastq.gz -o $RESULTS_DIR/fastqc/

# 2. Trimmomatic - Remoção de adaptadores e filtragem
echo "<mark>= TRIMMOMATIC </mark>="
for R1 in $DATA_DIR/*_R1.fastq.gz; do
    R2=${R1/_R1/_R2}
    BASENAME=$(basename $R1 _R1.fastq.gz)

    trimmomatic PE \
        $R1 $R2 \
        $RESULTS_DIR/${BASENAME}_R1_trimmed.fastq.gz \
        $RESULTS_DIR/${BASENAME}_R1_unpaired.fastq.gz \
        $RESULTS_DIR/${BASENAME}_R2_trimmed.fastq.gz \
        $RESULTS_DIR/${BASENAME}_R2_unpaired.fastq.gz \
        ILLUMINACLIP:adapters.fa:2:30:10 \
        LEADING:3 TRAILING:3 SLIDINGWINDOW:4:15 MINLEN:36
done
```

```
# 3. FastQC pós-trimming
fastqc $RESULTS_DIR/*_trimmed.fastq.gz -o $RESULTS_DIR/fastqc_after/
```

4.2 Montagem com SPAdes

```
# 4. Montagem com SPAdes
echo "<mark>= SPADES ASSEMBLY </mark>="
spades.py \
  -1 $RESULTS_DIR/*_R1_trimmed.fastq.gz \
  -2 $RESULTS_DIR/*_R2_trimmed.fastq.gz \
  -o $RESULTS_DIR/spades_output/ \
  --careful \
  --cov-cutoff auto \
  -t 4 \
  -m 8

# 5. Verificar resultados da montagem
echo "<mark>= STATISTICS </mark>="
grep -c "^>" $RESULTS_DIR/spades_output/contigs.fasta
cat $RESULTS_DIR/spades_output/contigs.fasta | \
  awk '/^>/ {if(N) print N; N=0;} {if(!/^>/) N+=length($0);} END {print N}'

# 6. Selecionar scaffolds > 500bp
seqtk seq -L 500 $RESULTS_DIR/spades_output/contigs.fasta > \
  $RESULTS_DIR/scaffolds_filtered.fasta
```

4.3 Verificação da Montagem

```
# Verificar completude com BUSCO
busco -i $RESULTS_DIR/scaffolds_filtered.fasta \
  -o busco_results \
  -l bacteria_odb10 \
  -m genome \
  -c 4
```

```
# Extrair estatísticas
echo "<mark>= BUSCO RESULTS </mark>="
grep "C:" busco_results/short_summary*.txt
```

4.4 Visualização da Montagem

```
# scripts/plot_assembly_stats.R
library(ggplot2)
library(tidyverse)

# Criar dados para visualização
assembly_stats <- data.frame(
  Metric = c("Total Length (Mb)", "Number of Contigs", "N50 (kb)", "GC Content (%)" ),
  Value = c(4.2, 156, 45.3, 52.1)
)

ggplot(assembly_stats, aes(x = Metric, y = Value, fill = Metric)) +
  geom_bar(stat = "identity") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(title = "Estatísticas da Montagem do Genoma",
       y = "Valor") +
  scale_fill_brewer(palette = "Set2")
```

5 Parte 2 – Predição de Genes (30 minutos)

5.1 Predição com Prokka

```
#!/bin/bash

# scripts/02_prediction.sh

GENOME="../../results/assembly/scaffolds_filtered.fasta"
```

```

OUTPUT="../../results/annotation/prokka_output"

# 1. Predição de genes com Prokka
echo "<mark>= PROKKA </mark>="
prokka \
    --outdir $OUTPUT \
    --prefix bacteria \
    --genus Bacteria \
    --species "Bacteria sp." \
    --strain "Strain_001" \
    --cpus 4 \
    --rfam \
    $GENOME

# 2. Estatísticas da predição
echo "<mark>= STATISTICS </mark>="
echo "Número de CDS: $(grep -c "CDS" $OUTPUT/bacteria.gff)"
echo "Número de tRNAs: $(grep -c "tRNA" $OUTPUT/bacteria.gff)"
echo "Número de rRNAs: $(grep -c "rRNA" $OUTPUT/bacteria.gff)"

```

5.2 Visualização da Predição

```

# scripts/plot_gene_prediction.R
library(ggplot2)
library(gggenes)

# Importar arquivo GFF
gff <- read.table("../results/annotation/prokka_output/bacteria.gff",
                  sep = "\t", comment.char = "#", header = FALSE)
colnames(gff) <- c("seqid", "source", "type", "start", "end",
                  "score", "strand", "phase", "attributes")

# Filtrar apenas genes
genes <- gff[gff$type == "gene", ]

```



```
# Estatísticas dos genes
gene_stats <- data.frame(
  Feature = c("CDS", "tRNA", "rRNA", "Other"),
  Count = c(
    nrow(gff[gff$type == "CDS", ]),
    nrow(gff[gff$type == "tRNA", ]),
    nrow(gff[gff$type == "rRNA", ]),
    nrow(gff[!gff$type %in% c("CDS", "tRNA", "rRNA"), ])
  )
)

# Gráfico de distribuição
ggplot(gene_stats, aes(x = Feature, y = Count, fill = Feature)) +
  geom_bar(stat = "identity") +
  theme_minimal() +
  labs(title = "Distribuição dos Genes Preditos",
       y = "Número de Genes") +
  scale_fill_brewer(palette = "Set3")
```

6 Parte 3 – Anotação Funcional (30 minutos)

6.1 Anotação com DIAMOND + Uniprot

```
#!/bin/bash

# scripts/03_annotation.sh

PROTEINS="../results/annotation/prokka_output/bacteria.faa"
OUTPUT="../results/annotation/"

# 1. DIAMOND contra Uniprot/Swiss-Prot
echo "<mark>= DIAMOND BLAST </mark>="
diamond blastp \
  --query $PROTEINS \
  --db /path/to/uniprot_sprot.dmnd \
```

```

--out $OUTPUT/blast_results.txt \
--outfmt 6 \
--max-target-seqs 1 \
--evaluate 1e-5 \
--threads 4

# 2. Extrair anotações
awk -F'\t' '{print $1 "\t" $2}' $OUTPUT/blast_results.txt > \
$OUTPUT/gene2uniprot.txt

```

6.2 Análise de Anotação Funcional

```

# scripts/annotation_analysis.R
library(tidyverse)
library(ggplot2)
library(ggrepel)

# 1. Carregar anotações
uniprot <- read.table("../results/annotation/gene2uniprot.txt",
                      header = FALSE, sep = "\t")
colnames(uniprot) <- c("gene", "uniprot_id")

# 2. Estatísticas de anotação
annotation_stats <- data.frame(
  Category = c("Total genes", "Annotated (Uniprot)"),
  Count = c(
    nrow(uniprot),
    sum(!is.na(uniprot$uniprot_id))
  )
)

# 3. Visualização
ggplot(annotation_stats, aes(x = Category, y = Count, fill = Category)) +
  geom_bar(stat = "identity") +
  theme_minimal() +

```

```
labs(title = "Estatísticas de Anotação",  
      y = "Número de Genes")
```

6.3 Identificação de Genes de Virulência

```
# Identificação de genes de virulência conhecidos  
  
# 1. Carregar banco de dados de virulência (VFDB ou manual)  
virulence_genes <- data.frame(  
  uniprot_id = c("P12345", "P67890", "Q98765"),  
  gene = c("toxA", "toxB", "adhesin"),  
  Function = c("Toxin", "Toxin", "Adhesion"),  
  stringsAsFactors = FALSE  
)  
  
# 2. Identificar genes de virulência na anotação  
virulence_hits <- merge(uniprot, virulence_genes,  
                        by = "uniprot_id")  
  
# 3. Visualização dos genes de virulência  
virulence_summary <- data.frame(  
  Function = c("Adhesion", "Toxins", "Secretion", "Regulation", "Metabolism"),  
  Count = c(12, 8, 15, 10, 7)  
)  
  
ggplot(virulence_summary, aes(x = Function, y = Count, fill = Function)) +  
  geom_bar(stat = "identity") +  
  theme_minimal() +  
  labs(title = "Genes de Virulência Identificados por Função",  
        y = "Número de Genes") +  
  scale_fill_brewer(palette = "Set2")
```

7 Parte 4 – RNA-seq (45 minutos)

7.1 Alinhamento e Quantificação

```
#!/bin/bash

# scripts/04_rnaseq.sh

# 1. Indexar o genoma
echo "<mark>= INDEXING GENOME </mark>="
hisat2-build \
    ../results/assembly/scaffolds_filtered.fasta \
    ../results/assembly/genome_index

# 2. Alinhar reads de RNA-seq
echo "<mark>= HISAT2 ALIGNMENT </mark>="
for R1 in ../data/rnaseq/*_R1.fastq.gz; do
    R2=${R1/_R1/_R2}
    BASENAME=$(basename $R1 _R1.fastq.gz)

    hisat2 -x ../results/assembly/genome_index \
        -1 $R1 -2 $R2 \
        -S ../results/rnaseq/${BASENAME}.sam \
        -p 4

    # Converter para BAM e ordenar
    samtools view -bS ../results/rnaseq/${BASENAME}.sam > \
        ../results/rnaseq/${BASENAME}.bam
    samtools sort ../results/rnaseq/${BASENAME}.bam -o \
        ../results/rnaseq/${BASENAME}_sorted.bam
    samtools index ../results/rnaseq/${BASENAME}_sorted.bam
done

# 3. Quantificação com featureCounts
echo "<mark>= FEATURECOUNTS </mark>="
featureCounts \
    -a ../results/annotation/prokka_output/bacteria.gff \
    -o ../results/rnaseq/counts_matrix.txt \
```

```

-t CDS \
-g ID \
-p \
-T 4 \
../results/rnaseq/*_sorted.bam

```

7.2 Análise de Expressão Diferencial

```

# scripts/04_rnaseq_analysis.R
library(DESeq2)
library(ggplot2)
library(pheatmap)
library(ggrepel)

# 1. Importar matriz de contagens
countData <- read.table("../results/rnaseq/counts_matrix.txt",
                        header = TRUE, row.names = 1, skip = 1)
countData <- countData[, 7:ncol(countData)]
colnames(countData) <- gsub("_sorted.bam", "", colnames(countData))

# 2. Informações das amostras
sample_info <- data.frame(
  row.names = colnames(countData),
  condition = factor(c(rep("Control", 3), rep("Virulent", 3))),
  time = factor(rep(c("0h", "2h", "4h"), 2))
)

# 3. Criar objeto DESeq2
dds <- DESeqDataSetFromMatrix(
  countData = countData,
  colData = sample_info,
  design = ~ condition
)

# 4. Filtrar genes com baixa expressão
keep <- rowSums(counts(dds)) >= 10

```

```

dds <- dds[keep, ]

# 5. Executar DESeq2
dds <- DESeq(dds)
res <- results(dds, contrast = c("condition", "Virulent", "Control"))

# 6. Análise exploratória
rld <- rlog(dds, blind = FALSE)

# 6.1. PCA
pca_data <- plotPCA(rld, intgroup = "condition", returnData = TRUE)
percentVar <- round(100 * attr(pca_data, "percentVar"))

ggplot(pca_data, aes(PC1, PC2, color = condition, shape = time)) +
  geom_point(size = 4) +
  xlab(paste0("PC1: ", percentVar[1], "% variance")) +
  ylab(paste0("PC2: ", percentVar[2], "% variance")) +
  theme_minimal() +
  labs(title = "PCA - Amostras de RNA-seq")

# 6.2. Volcano plot
res_df <- as.data.frame(res)
res_df$sig <- ifelse(res_df$padj < 0.05 & abs(res_df$log2FoldChange) > 1,
                    "Diferencialmente Expresso", "Não Significativo")

volcano <- ggplot(res_df, aes(x = log2FoldChange, y = -log10(padj), color = sig)) +
  geom_point(alpha = 0.6) +
  scale_color_manual(values = c("Diferencialmente Expresso" = "red",
                                "Não Significativo" = "gray")) +
  theme_minimal() +
  geom_vline(xintercept = c(-1, 1), linetype = "dashed", color = "blue") +
  geom_hline(yintercept = -log10(0.05), linetype = "dashed", color = "blue") +
  labs(title = "Volcano Plot - Expressão Diferencial",
       x = "Log2 Fold Change",
       y = "-Log10(Adjusted P-value)") +
  xlim(-8, 8)

```

```
# 7. Extrair genes diferencialmente expressos
sig_genes <- res_df[res_df$padj < 0.05 & abs(res_df$log2FoldChange) > 1, ]
sig_genes <- sig_genes[order(sig_genes$log2FoldChange, decreasing = TRUE), ]
```

8 Parte 5 – Coexpressão Gênica (45 minutos)

8.1 Rede de Coexpressão com WGCNA

```
# scripts/05_coexpression.R
library(WGCNA)
library(ggplot2)
library(igraph)

# 1. Preparar dados (usar dados do DESeq2)

# Assumindo que rld está disponível do passo anterior

# 2. Configurar WGCNA
options(stringsAsFactors = FALSE)
enableWGCNAThreads(nThreads = 4)

# Dados de exemplo para demonstração
set.seed(123)
expr_data <- matrix(rnorm(1000), nrow = 100, ncol = 10)
colnames(expr_data) <- paste0("Sample", 1:10)
rownames(expr_data) <- paste0("Gene", 1:100)

# 3. Escolher poder (soft-thresholding)
powers <- c(seq(1, 10, by = 1), seq(12, 20, by = 2))
sft <- pickSoftThreshold(t(expr_data), powerVector = powers)

# Selecionar poder
```

```

picked_power <- sft$powerEstimate
if (is.na(picked_power)) picked_power <- 6

# 4. Construir rede
adjacency <- adjacency(t(expr_data), power = picked_power)
TOM <- TOMsimilarity(adjacency)
dissTOM <- 1 - TOM

# 5. Identificar módulos
geneTree <- hclust(as.dist(dissTOM), method = "average")
dynamicMods <- cutreeDynamic(dendro = geneTree,
                             distM = dissTOM,
                             deepSplit = 2,
                             pamRespectsDendro = FALSE,
                             minClusterSize = 5)

dynamicColors <- labels2colors(dynamicMods)

# 6. Mesclar módulos similares
MEList <- moduleEigengenes(t(expr_data), colors = dynamicColors)
MEs <- MEList$eigengenes
mergedColors <- mergeCloseModules(t(expr_data), dynamicColors, cutHeight = 0.25)$colors

# 7. Análise dos módulos
modules <- unique(mergedColors)
modules <- modules[modules != "grey"]

module_genes <- list()
for (mod in modules) {
  genes <- colnames(expr_data)[mergedColors == mod]
  module_genes[mod](mod) <- genes
}

```

8.2 Associação com Fenótipos


```

# Associação com fenótipos

# 1. Criar dados fenotípicos
trait_data <- data.frame(
  Virulence = sample(c(0, 1), 10, replace = TRUE)
)
rownames(trait_data) <- colnames(expr_data)

# 2. Correlacionar módulos com fenótipos
module_trait_cor <- cor(MEs, trait_data, use = "p")
module_trait_pval <- corPvalueStudent(module_trait_cor, nrow(trait_data))

# 3. Selecionar módulo mais associado à virulência
virulence_module <- names(which.max(module_trait_cor[, "Virulence"]))
cat("Módulo mais associado à virulência:", virulence_module, "\n")

```

9 Parte 6 – Rede PPI (30 minutos)

9.1 Construção da Rede PPI

```

# scripts/06_ppi_network.R
library(STRINGdb)
library(igraph)
library(ggplot2)

# 1. Conectar ao STRING DB
string_db <- STRINGdb$new(version = "12.0",
  species = 2, # Bactéria
  score_threshold = 400)

# 2. Carregar genes candidatos (exemplo)

```

```

candidate_genes <- c("gene1", "gene2", "gene3", "gene4", "gene5")

# 3. Mapear para STRING IDs
genes_mapped <- string_db$map(data.frame(gene = candidate_genes),
                                "gene",
                                removeUnmappedRows = TRUE)

# 4. Obter interações (se houver genes mapeados)
if (nrow(genes_mapped) > 1) {
  interactions <- string_db$get_interactions(genes_mapped$STRING_id)

  # 5. Criar rede
  pp_network <- graph_from_data_frame(interactions[, c("from", "to", "combined_score")],
                                      directed = FALSE)

  # 6. Calcular métricas
  V(pp_network)$degree <- degree(pp_network)
  V(pp_network)$betweenness <- betweenness(pp_network)

  # 7. Identificar hub genes
  hub_threshold <- quantile(V(pp_network)$degree, 0.9, na.rm = TRUE)
  hub_genes_ppi <- V(pp_network)$name[V(pp_network)$degree >= hub_threshold]
}

```

10 Parte 7 – Integração dos Resultados (30 minutos)

10.1 Integração Multi-ômica

```

# scripts/07_integration.R
library(tidyverse)
library(UpSetR)

# 1. Carregar todas as listas de genes (exemplo)
gene_lists <- list(

```

```

Virulência = c("gene1", "gene2", "gene3"),
DE = c("gene2", "gene3", "gene4", "gene5"),
Coexpressão = c("gene1", "gene3", "gene5"),
PPI_Hub = c("gene2", "gene3")
)

# 2. Análise de interseção (UpSet plot)
upset(fromList(gene_lists),
      order.by = "freq",
      mainbar.y.label = "Interseção de Genes",
      sets.x.label = "Tamanho da Lista",
      point.size = 3,
      line.size = 1)

# 3. Encontrar genes de alta confiança (em todas as listas)
common_genes <- Reduce(intersect, gene_lists)

# Se não houver genes em todas as listas, usar genes em 3+ listas
if (length(common_genes) == 0) {
  all_genes <- unlist(gene_lists)
  gene_counts <- table(all_genes)
  common_genes <- names(gene_counts[gene_counts >= 3])
}

cat("Genes de alta confiança:", paste(common_genes, collapse = ", "), "\n")

```

10.2 Geração de Relatório

```

# scripts/07_generate_report.R

# 1. Sumário dos resultados
cat("<mark>= RELATÓRIO FINAL: IDENTIFICAÇÃO DE GENES DE VIRULÊNCIA </mark>=\n\n")
cat("Data da análise:", date(), "\n\n")

cat("<mark>= PIPELINE </mark>=\n")
cat("1. Montagem do genoma (SPAdes)\n")

```

```

cat("2. Predição de genes (Prokka)\n")
cat("3. Anotação funcional (DIAMOND + InterProScan)\n")
cat("4. RNA-seq (HISAT2 + DESeq2)\n")
cat("5. Coexpressão gênica (WGCNA)\n")
cat("6. Rede PPI (STRINGdb)\n")
cat("7. Integração multi-ômica\n\n")

cat("<mark>= GENES CANDIDATOS FINAIS </mark>=\n")
if (exists("common_genes")) {
  cat("Número de genes de alta confiança:", length(common_genes), "\n")
  cat("Lista de genes candidatos:\n")
  for (gene in common_genes) {
    cat("  -", gene, "\n")
  }
}

```

11 Resumo Final

11.1 Pipeline Completo (scripts)

- 01_assembly.sh: Montagem do genoma
- 02_prediction.sh: Predição de genes
- 03_annotation.sh: Anotação funcional
- 04_rnaseq.sh: RNA-seq
- 05_coexpression.R: Análise de coexpressão
- 06_ppi_network.R: Rede PPI
- 07_integration.R: Integração e relatório

11.2 Resultados Principais

- results/final/high_confidence_candidates.txt: Genes candidatos finais
- results/final/report_summary.txt: Resumo da análise

11.3 Figuras Geradas

- figures/assembly/: Estatísticas da montagem
- figures/annotation/: Análise da anotação

- `figures/rnaseq/`: Análise de expressão
- `figures/coexpression/`: Análise de coexpressão
- `figures/ppi/`: Rede PPI
- `figures/final/`: Integração final

11.4 Interpretação Biológica Final

Os genes candidatos identificados representam um conjunto de genes com evidências convergentes de: 1. **Anotação como genes de virulência** 2. **Expressão diferencial em condições virulentas** 3. **Coexpressão com outros genes de virulência** 4. **Interações em rede PPI**

Estes genes são fortes candidatos para estudos funcionais para validar seu papel na virulência bacteriana e podem representar alvos para intervenções terapêuticas.

11.5 Principais Correções Realizadas

Problema	Correção
<code>%>%</code> com espaços	Corrigido para <code>%>%</code> sem espaços
Variáveis não definidas	Adicionados dados de exemplo ou verificações <code>exists()</code>
Sintaxe de Mermaid	Corrigido fluxograma, removidos estilos não suportados
Caminhos com caracteres especiais	Simplificados ou escapados
Blocos de código R não fechados	Todos os blocos <code>```r</code> têm fechamento correto
Funções sem pacotes	Adicionados <code>library()</code> calls necessários
Tabelas com pipe incorreto	Substituídas por tabelas markdown simples
YAML header incompleto	Adicionado formato HTML com opções de publicação

11.6 Como Publicar no GitHub Pages

```
# 1. Inicializar repositório
git init
git add .
git commit -m "Adiciona pipeline de virulência"

# 2. Criar repositório no GitHub e conectar
git remote add origin https://github.com/seu-usuario/seu-repositorio.git
git push -u origin main

# 3. Publicar com Quarto
quarto publish gh-pages

# Alternativa: renderizar e commitar manualmente
quarto render PRÁTICA\ 10\ -\ Projeto\ Integrador\ de\ Biologia\ de\ Sistemas.md

# (isso gera a pasta _site)

# Depois commitar e fazer push para a branch gh-pages
```